

ECE456 Network Centric Programming – Midterm

1. (5 points) Dead code and optimization.

Consider the following program compiled **without** optimization:

```
1 int main() {
2     long long sum = 0;
3     for (long long i = 0; i < 1000000000LL; i++) {
4         sum += i;
5     }
6     return 0;
7 }
```

On this machine the program takes **1.5 seconds** to run.

If we compile with optimization how long does it take (rounded to the nearest 0.1 seconds)?

Time: _____ seconds

```
1 int main() {
2     long long sum = 0;
3     for (long long i = 0; i < 1'000'000'000LL; i++) {
4         sum += i;
5     }
6     cout << "sum=" << sum << endl;
7     return 0;
8 }
```

Time: _____ seconds

Now consider:

```
1 #include <unistd.h>
2
3 int main() {
4     for (int i = 0; i < 1000; i++) {
5         usleep(1000); // 1000 microseconds = 1 ms
6     }
7     return 0;
8 }
```

Roughly how long does this program take to run (assume `usleep()` is exact)?

Time: _____ seconds

Next:

```
1 int main() {
2     int a, b;
3     scanf("%d %d", &a, &b);
4     int c = a + b;
5     return 0;
6 }
```

At runtime, is an integer addition performed? _____

Finally, consider:

```

1 int add(int a, int b);
2
3 int main() {
4     int c = add(2, 3);
5     return 0;
6 }
7
8 // this function is in a different file
9 int add(int a, int b) {
10     return a + b;
11 }

```

With normal compilation (no `inline`), is an addition performed at runtime? _____

If we mark `add()` as `inline` and enable optimization, can the compiler remove the addition and replace it with a constant? _____

2. (5 points) For each program, write what happens.

```

1 #include <sys/stat.h>
2 #include <sys/types.h>
3 #include <iostream>
4 using namespace std;
5 int main() {
6     int status = chdir("xyz");
7     cout << "status: " << status << endl;
8     return 0;
9 }

```

Net effect of the program: _____

```

1 #include <sys/stat.h>
2 #include <sys/types.h>
3 #include <iostream>
4 using namespace std;
5 int main() {
6     int status = mkdir("xyz", 0755);
7     cout << "status: " << status << endl;
8     return 0;
9 }

```

Net effect of the program: _____

```

1 #include <sys/stat.h>
2 #include <sys/types.h>
3 #include <iostream>
4 #include <fstream>
5 #include <unistd.h>
6 using namespace std;
7 int main() {
8     char filename[10] = "x";
9     for (int i = 1; i <= 3; i++) {
10         filename[1] = '0' + i;
11         filename[2] = '\0';
12         int status = mkdir(filename, 0755);
13         cout << "status: " << status << endl;
14         status = chdir(filename);
15         cout << "status: " << status << endl;
16         ofstream f("yo!");
17         f << "hello" << endl;
18         f.close();
19         status = chdir("..");
20         cout << "status: " << status << endl;
21     }
22     return 0;
23 }

```

Net effect of the program: _____

3. (5 points) What does the following command do?

```
1 kill -15 <pid>
2
3 kill -9 <pid>
```

4. (5 points) What happens

```
1 #include <iostream>
2 #include <thread>
3 using namespace std;
4
5 void f() {
6     cout << "hello" << endl;
7 }
8
9 void g() {
10    cout << "world" << endl;
11 }
12
13 int main() {
14     thread t1(f);
15     thread t2(g);
16     t1.join();
17     t2.join();
18     return 0;
19 }
```

What is printed: _____

5. (5 points) Fill in the missing line to send a short message.

```
1 #include <sys/socket.h>
2 int main() {
3     int sock = socket(AF_INET, SOCK_STREAM, 0);
4     const char* msg = "hello";
5     _____(sock, msg, 5, 0);
6 }
```

Is the message guaranteed to be sent?

Is the message guaranteed to be received?

6. (5 points) What condition indicates an error from `recv()`?

```
1 int n = recv(sock, buf, 1000, 0);
```

7. (5 points) What is received by the server in the following code?

Client:

```
1 int n = send(sock, "ABC", 3, 0);
2 n = send(sock, "DEF", 3, 0);

1 char buf[1000];
2 int n = recv(sock, buf, 1000, 0);
```

What is n assuming:

The sends fail? _____ The sends succeed? _____

8. (5 points) Messages on a network are sent in chunks called _____.

If you send a message broken down into pieces, are they guaranteed to be received in order? _____

If you send a message via socket, are the bytes received guaranteed to be the same as the bytes sent? _____

9. (10 points) Fill in the missing code so the program sends the entire buffer.

```

1  int sent = 0;
2
3  while(sent < len) {
4
5      int n = send(sock,
6                  buf + sent,
7                  len - sent,
8                  0);
9
10     if(n < 0)
11         return -1;
12
13     -----;
14 }

```

10. (10 points) Fill in the missing code so the program receives a full message.

```

1  int received = 0;
2
3  while(received < len) {
4
5      int n = recv(sock,
6                  buf + received,
7                  len - received,
8                  0);
9
10     if(n < 0)
11         return -1;
12
13     -----;
14 }

```

11. (8 points) What does the following IP address represent?

127.0.0.1

Short answer.

12. (8 points) Which protocol guarantees reliable delivery?

- UDP
- TCP

13. (10 points) A TCP client sends a message.

The client later receives an ****ACK**** packet.

Does this guarantee that the server application has processed the message?

- Yes
- No

14. (10 points) What layer generates the ACK packet?

- Application
- TCP
- IP

15. (12 points) Consider the following server code.

```
1 int client = accept(server_fd, NULL, NULL);
2
3 char buf[100];
4
5 recv(client, buf, 100, 0);
6
7 printf("%s\n", buf);
```

Is it guaranteed that the full message was received?

- Yes
- No

Brief reason:

16. (12 points) What is wrong with the following code?

(single phrase)

```
1 char buf[100];
2
3 int n = recv(sock, buf, 100, 0);
4
5 printf("%s\n", buf);
```

Answer:

17. (15 points) Write a loop that reads from a socket until the connection closes.

Use:

- recv
- break when recv returns 0

18. (10 points) Write a program to open a file and write a multiplication table using a nested loop.
19. (10 points) Write a program that creates a subprocess using `fork()`. The parent should print "parent" and the child should print "child".
20. (10 points) Write a program that creates 2 threads and prints "hello" and goodbye 10 times.